

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Penganter Mikrokontroler

Christopher Joenathan Hindarto - 5024241028

07 November 2026

1 Pendahuluan

1.1 Latar Belakang

Pengembangan sistem tertanam (*embedded system*) modern menuntut pemahaman mendalam mengenai arsitektur perangkat keras serta pemilihan *software stack* dan lingkungan pengembangannya. Perkembangan platform mikrokontroler seperti STM32 Blackpill berbasis ARM Cortex-M4 dan ESP32 berbasis Xtensa LX6 dual-core menawarkan fleksibilitas eksekusi program yang bervariasi. Penggunaan IDE terintegrasi seperti Visual Studio Code dengan ekstensi PlatformIO mempermudah pengelolaan berbagai *framework* (seperti Arduino dan ESP-IDF) dalam satu ekosistem terpadu. Penggunaan *framework* Arduino menyajikan abstraksi tingkat tinggi untuk kemudahan prototipe, sedangkan *framework* native seperti ESP-IDF menyajikan kontrol berdaya tinggi hingga ke tingkat *task scheduling* FreeRTOS dan registri sistem.

Dalam perancangan antarmuka perangkat keras, pengelolaan sinyal Masukan/Keluaran Serbaguna atau *General Purpose Input/Output* (GPIO) memainkan peranan utama. Namun, transisi sinyal digital pada input mekanis seperti tombol tekan (*pushbutton*) tidak terlepas dari fenomena getaran fisik kontak mekanik (*bouncing*) yang memicu deret pulsa transisi cepat HIGH-LOW sebelum sinyal benar-benar stabil. Jika dibiarkan tanpa penanganan, hal ini terbaca sebagai penekanan ganda oleh mikroprosesor. Selain itu, kebutuhan pembacaan sinyal kontinu menuntut pemanfaatan periferal *Analog-to-Digital Converter* (ADC). Oleh karena itu, praktikum ini dilaksanakan untuk memberikan pemahaman fundamental mengenai integrasi arsitektur board, manajemen GPIO digital-analog, serta penanganan *debouncing* sinyal sebelum membangun sistem kontrol dan IoT yang lebih kompleks.

1.2 Dasar Teori

GPIO dapat dikonfigurasikan ke dalam berbagai mode operasi dasar, yaitu *digital output*, *digital input*, serta *analog input*. Pada mode *digital input*, keterikatan sinyal terhadap noise eksternal atau status menggantung (*floating*) dapat dicegah dengan mengimplementasikan mekanisme resistor *pull-up* atau *pull-down*, baik secara eksternal pada papan reka (*breadboard*) maupun memanfaatkan fungsi *pull internal* pada mikrokontroler. Penerapan konfigurasi ini menjamin kepastian logika tegangan (HIGH atau LOW) saat saklar mekanis tidak diaktifkan. Untuk mengatasi masalah *bouncing* pada input mekanis, teknik penyaringan sinyal secara komputasional (*debouncing*) wajib diimplementasikan dengan memvalidasi durasi kestabilan sinyal menggunakan pewaktu sistem (seperti fungsi `millis()` atau pewaktu *tick* FreeRTOS) sebelum mengonfirmasi status logis tombol.

Berbeda dengan mode digital yang hanya membedakan dua kondisi status tegangan diskrit (HIGH/LOW), mode *analog input* menggunakan *Analog-to-Digital Converter* (ADC) untuk membaca besaran tegangan analog yang berubah secara kontinu. ADC mengubah tegangan analog (seperti keluaran sensor resistif pembagi tegangan LDR pada rentang 0–3.3V) menjadi representasi nilai kuantisasi digital. Sebagai contoh, resolusi ADC 12-bit pada ESP32 membagi rentang tegangan referensi menjadi $2^{12} = 4096$ tingkatan langkah (*step/levels*), sehingga menghasilkan nilai mentah digital dari 0 hingga 4095. Hal ini memungkinkan mikrokontroler untuk mengukur dan memproses besaran fisis secara lebih presisi.

2 Tugas Pendahuluan

2.1 Soal 1: Perbedaan Framework Arduino dan ESP-IDF

2.1.1 Perbedaan Mendasar

Framework **Arduino** dirancang sebagai *abstraction layer* tingkat tinggi (high-level abstraction) yang mengisolasi pemrogram dari detail arsitektur hardware mikrokontroler. Kode ditulis dalam format simplified C++ dengan paradigma *Hardware Abstraction Layer* (HAL) yang homogen across multiple platform (AVR, ESP32, STM32, ARM). Sebaliknya, **ESP-IDF (Espressif IoT Development Framework)** adalah *official Software Development Kit* (SDK) dari Espressif Systems yang berbasis sistem operasi real-time FreeRTOS. ESP-IDF bekerja pada tingkat rendah (low-level access) dan memberikan akses langsung terhadap register hardware, konfigurasi dual-core, manajemen daya, memori DMA, dan arsitektur pengamanan hardware.

2.1.2 Tabel Perbandingan

Fitur / Parameter	Arduino Framework	ESP-IDF Framework
Arsitektur Utama	Monolithic (single-loop execution model)	FreeRTOS-based multi-threading / task scheduling
Abstraksi Hardware	Sangat tinggi, universal API	Rendah hingga menengah, terspesifikasi untuk ESP32
Manajemen Memori	Otomatis/Tersembunyi	Manual / Fine-grained control (SRAM, PSRAM, Heap)
Multi-core Support	Terbatas / Perlu wrapper khusus	Native support untuk kontrol Core 0 & Core 1
Ukuran Binary	Lebih besar (overhead bawaan library)	Terkompresi dan efisien (hanya library terpakai)

2.1.3 Kelebihan dan Kekurangan

- **Kelebihan Arduino Framework:**
  - *Learning curve* sangat landai, ramah untuk pemula.
  - Ekosistem pustaka (libraries) sangat luas dan cepat dalam prototyping.
  - Portabilitas kode tinggi antar platform mikrokontroler.
- **Kekurangan Arduino Framework:**
  - Efisiensi eksekusi dan kontrol memori relatif rendah.
  - Sulit memanfaatkan fitur spesifik ESP32 secara optimal (seperti deep sleep fine control, protokol Wi-Fi/Bluetooth kustom).
  - Pengolahan tugas berikatan waktu (*time-critical tasks*) terhambat oleh sifat siklus `loop()`.
- **Kelebihan ESP-IDF Framework :**
  - Performa maksimal dan optimasi memori yang sangat presisi.

  - Dukungan penuh terhadap FreeRTOS (Task management, Queue, Mutex, Semaphore).
  - Konfigurasibilitas tinggi melalui sistem `menuconfig` (Kconfig).
- **Kekurangan ESP-IDF Framework :**
  - *Kurva pembelajaran* sangat curam dan kompleksitas struktur kode tinggi.
  - Memerlukan waktu pengembangan yang lebih lama (*development time*).
  - Tidak kompatibel secara langsung dengan library standar Arduino tanpa kompilasi komponen khusus.

2.2 Soal 2: Pemrograman STM32 Blackpill vs ESP32 dan Peran USB-to-Serial

2.2.1 Perbedaan Jalur Programming

Board **ESP32** (seperti ESP32 NodeMCU atau DevKit v1) secara umum dapat langsung diprogram via kabel USB ke PC karena di dalam papan pengembang tersebut sudah terintegrasi IC **USB-to-Serial/UART Converter** (contoh: CP2102, CH340, atau FT232R) serta **rangkaian auto-reset / auto-bootloader** (biasanya terdiri dari dua buah transistor NPN).

Sebaliknya, **STM32F4xx Blackpill** secara default tidak dilengkapi dengan chip dedicated converter USB-to-Serial. Meskipun chip STM32 memiliki port USB native pada pin PA11 (USB DM) dan PA12 (USB DP), ROM bootloader pabrikan STM32 memerlukan interface pengunggahan khusus seperti SWD (Serial Wire Debug) atau memerlukan prosedur instalasi *custom USB bootloader* (seperti STM32duino bootloader) sebelum dapat diflash via USB langsung.

2.2.2 Peran Chip USB-to-Serial

Mikrokontroler pada dasarnya berkomunikasi melalui protokol serial bawaan yaitu **UART (Universal Asynchronous Receiver-Transmitter)** yang menggunakan level tegangan logika TTL (0V – 3.3V/5V). Sementara itu, PC/Laptop berkomunikasi via pengontrol protokol **USB (Universal Serial Bus)**.

Chip USB-to-Serial berfungsi sebagai **jembatan penerjemah (protocol bridge)** antara sinyal paket USB pada PC dan paket UART pada mikrokontroler.

2.2.3 Mekanisme Auto-Bootloading ESP32

Saat proses *flashing* pada ESP32 dilakukan dari komputer:

(a) Komputer mengirim sinyal kendali DTR (*Data Terminal Ready*) dan RTS (*Request to Send*) melalui driver USB.

(b) Chip USB-to-Serial mengubah sinyal ini menjadi logika pin fisiknya.

(c) Rangkaian otomatisasi pada papan ESP32 memanipulasi pin `EN` (Enable/Reset) dan `GPIO0` (Boot selection pin):

- Sinyal dikondisikan agar `GPIO0` berada pada level **LOW** saat `EN` mengalami transisi dari **LOW ke HIGH** (Reset sequence).

(d) Hal ini secara otomatis memaksa prosesor ESP32 masuk ke modul ROM Download Bootloader tanpa intervensi manual dari pengguna.

Pada STM32 Blackpill, untuk memprogram via SWD, programmer eksternal **ST-Link v2** terhubung langsung ke pin SWDIO (Serial Wire Data Input/Output), SWCLK (Serial Wire Clock), GND, dan 3.3V. ST-Link menggunakan protokol debugging ARM CoreSight yang dapat langsung menulis ke memori Flash mikrokontroler tanpa bergantung pada protokol UART bootloader.

2.3 Soal 3: Konfigurasi Pull-Up dan Pull-Down Resistor

Resistor *pull-up* dan *pull-down* digunakan untuk memastikan pin input digital mikrokontroler berada pada kondisi logika terdefinisi (definite state) ketika tidak terhubung ke beban aktif (mencegah kondisi *floating*).

2.3.1 Prinsip Kerja dan Kondisi Logika

Konfigurasi	Kondisi Tombol: TIDAK DITEKAN	Kondisi Tombol: DITEKAN
Pull-Up Resistor	**HIGH** (VCC / 3.3V atau 5V) *Pin terhubung ke VCC melalui resistor, arus dari VCC masuk ke pin.*	**LOW** (GND / 0V) *Arus mengalir langsung dari VCC melalui resistor menuju ground via tombol.*
Pull-Down Resistor	**LOW** (GND / 0V) *Pin terhubung ke Ground melalui resistor, tidak ada arus mengalir.*	**HIGH** (VCC / 3.3V atau 5V) *Pin terhubung langsung ke VCC melalui tombol.*

2.3.2 Rangkaian dan Aliran Arus

- **Pada Pull-Up:** Tombol dipasang antara Pin Digital dan GND. Resistor dipasang antara Pin Digital dan VCC. Logika bersifat *Active LOW*.
- **Pada Pull-Down:** Tombol dipasang antara Pin Digital dan VCC. Resistor dipasang antara Pin Digital dan GND. Logika bersifat *Active HIGH*.

2.4 Soal 4: Fenomena Bouncing dan Solusi De-bouncing

2.4.1 Penyebab Bouncing

Bouncing (bounce saklar mekanik) adalah fenomena terjadinya transisi logika sinyal mendadak yang berulang secara cepat (berupa osilasi High-Low berulang dalam orde milidetik) pada saat saklar atau tombol tekan (*push button*) dibuka atau ditutup.

Hal ini terjadi karena secara fisik, kontak mekanik internal pada saklar terbuat dari plat logam lentur. Ketika dua konduktor logam saling berbenturan, gaya elastisitas bahan menyebabkan kontak mengalami benturan kecil berturut-turut (*microscopic mechanical vibration*) sebelum akhirnya benar-benar menempel sempurna. Mikrokontroler yang eksekusinya sangat cepat akan membaca deretan osilasi ini sebagai penekanan tombol berulang kali.

2.4.2 Pendekatan untuk Mengatasi Bouncing (Debouncing)

1. Pendekatan Hardware (Hardware Debouncing) Menggunakan komponen elektronika pasif/aktif untuk memfilter sinyal osilasi sebelum masuk ke pin input mikrokontroler.
    - **Filter Low-Pass RC (Resistor-Capacitor):** Memasang kapasitor (misal 100 nF) secara paralel dengan tombol dan resistor penahan. Kapasitor berfungsi menyerap lonjakan tegangan cepat dan meredam osilasi logika.
    - **Schmitt Trigger (IC 74HC14):** Menggabungkan filter RC dengan gerbang Schmitt Trigger yang memiliki sifat *hysteresis* untuk mengubah transisi sinyal analog yang lambat atau berisolasi menjadi sinyal konstan digital bertransisi tajam.
2. Pendekatan Software (Software Debouncing) Menggunakan algoritma pada kode pemrogram untuk mengabaikan perubahan sinyal dalam durasi waktu tertentu.
    - **Delay-Based Method (Simple Delay):** Saat terdeteksi perubahan status pin, program melakukan jeda penundaan (misalnya `delay(20)` ms) untuk menunggu agar getaran mekanik selesai sebelum melakukan pembacaan ulang status pin.
    - **Millis / Timer-Based Non-Blocking Method:** Menyimpan data waktu (`millis()`) saat transisi pertama terjadi dan memastikan pembacaan status baru diproses hanya apabila selisih waktu dari transisi sebelumnya telah melebihi batas *debounce threshold* (misal 20–50 ms).

2.5 Soal 5: Prinsip Kerja ADC dan Pengaruh Resolusi Bit

2.5.1 Prinsip Kerja ADC

Analog-to-Digital Converter (ADC) mengonversi sinyal input tegangan kontinu (analog) menjadi nilai numerik diskrit (digital). Proses konversi ADC melibatkan tiga tahapan utama:

(a) **Sampling (Pencuplikan):** Sinyal analog kontinu dicuplik nilainya pada interval waktu tertentu.

(b) **Quantization (Kuantisasi):** Nilai tegangan kontinu hasil cuplikan dikelompokkan ke dalam tingkat-tingkat tegangan diskrit (*quantization levels*) terdekat sesuai batas tatanan referensi tegangan (V_{ref}).

(c) **Encoding (Pengkodean):** Nilai diskrit tersebut dikonversi menjadi biner sesuai besaran resolusi ADC.

Persamaan matematis pemetaan hasil konversi ADC dirumuskan sebagai:

$$\text{ADC Out} = \left\lfloor \frac{V_{in}}{V_{ref}} \times (2^N - 1) \right\rfloor \tag{1}$$

Di mana N adalah resolusi bit ADC, V_{in} adalah tegangan input analog, dan V_{ref} adalah tegangan referensi sistem (misal 3.3V).

2.5.2 Pengaruh Resolusi Bit terhadap Presisi dan Nilai LSB

Resolusi bit (N) menentukan jumlah total tingkatan kuantisasi yang dimiliki oleh ADC, yaitu sejumlah 2^N kombinasi nilai.

Presisi pembacaan ditentukan oleh nilai **Least Significant Bit (LSB)** atau *step resolution*, yang dirumuskan sebagai:

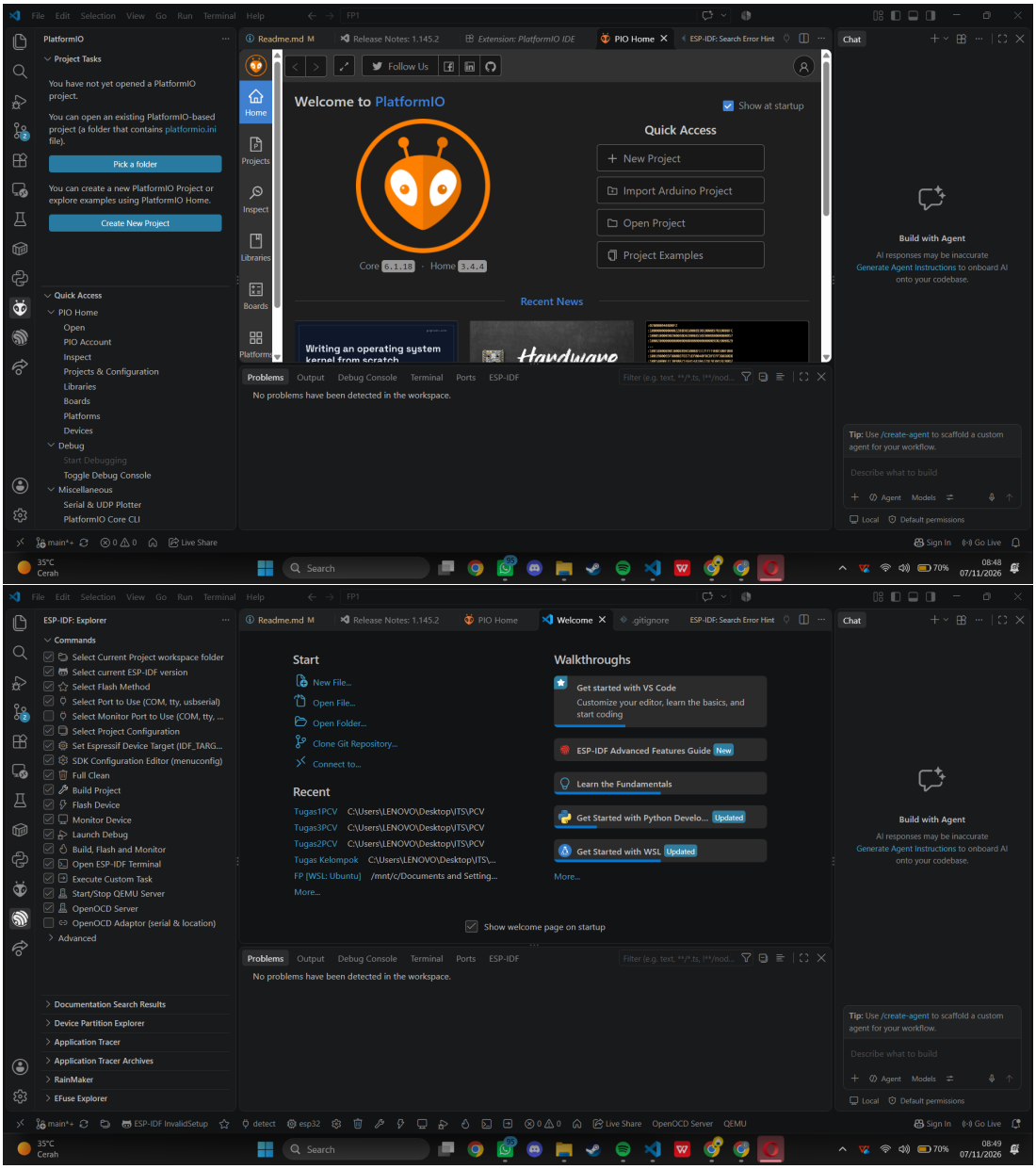
$$\text{LSB Step Voltage} = \frac{V_{ref}}{2^N - 1} \tag{2}$$

- **Makin tinggi resolusi bit (N):** Maka rentang nilai output digital (2^N) semakin besar, nilai step tegangan terkecil (LSB) semakin kecil, sehingga ketelitian/presisi pembacaan terhadap perubahan sinyal analog sekecil apa pun akan menjadi semakin mendalam dan akurat.
- **Perbandingan Resolusi:**
  - **ADC 10-bit ($2^{10} = 1024$ step):** Pada $V_{ref} = 3.3$V, nilai per-step (LSB) adalah $\frac{3.3\text{V}}{1023} \approx 3.22$ mV.
  - **ADC 12-bit ($2^{12} = 4096$ step):** Pada $V_{ref} = 3.3$V, nilai per-step (LSB) adalah $\frac{3.3\text{V}}{4095} \approx 0.805$ mV.

2.6 Soal 6: Bukti Setup Environment Tools (VSCode, PlatformIO, ESP-IDF, STM32, Serial Plotter)

Berikut adalah bukti tangkapan layar pengesetulangan (*setup verification*) perangkat lunak pengembangan Visual Studio Code beserta seluruh ekstensi yang dipersyaratkan.

2.7 Tangkapan Layar Setup PlatformIO, Framework ESP-IDF & STM32



Gambar 1: Bukti penginstalan ekstensi PlatformIO di VSCode beserta platform ESP32 (ESP-IDF) dan ST STM32.